

Transition: DS + Algorithms

Big-O
+ Recursion



First:

worksheets from last time?

Next: Confusing readings!

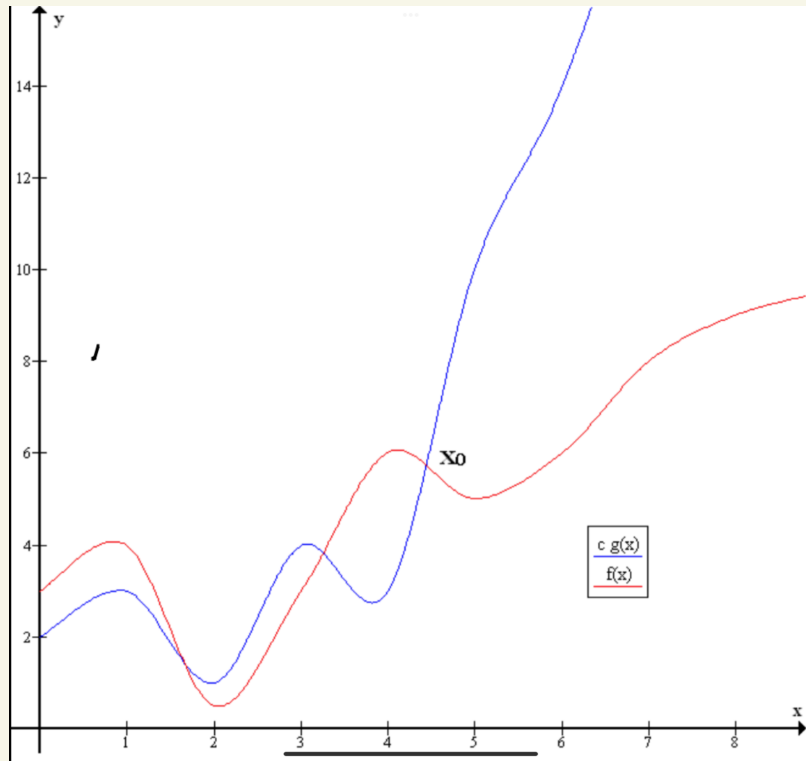
Let's pause on recursion -

I'll post something else covering
it soon.

Back to Big- O & why we care!

Asymptotic Analysis

How can we compare functions?



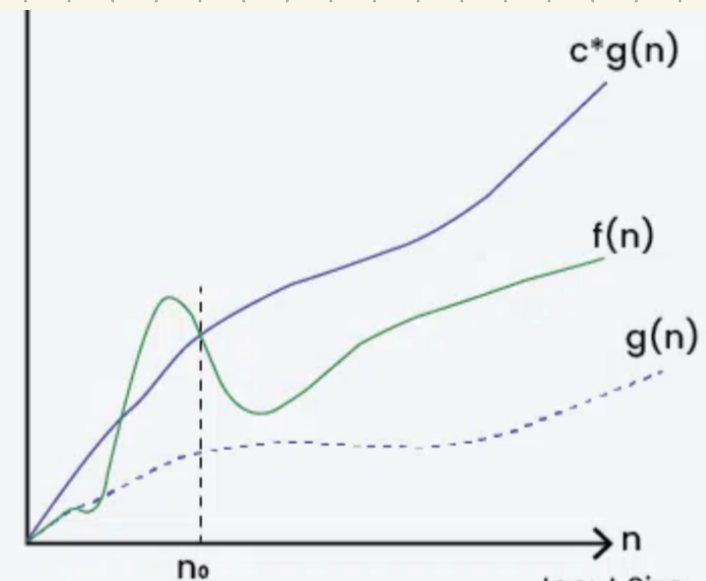
Big-O

Let f and g be functions from

$\mathbb{Z}^+$ to $\mathbb{R}^+$
why?

We say $f(n) = O(g(n))$ if $\exists$
constants c and n_0 s.t. $\forall n > n_0$,
 $f(n) \leq c \cdot g(n)$

Note: $O(g(n))$ is
a set.



Example

$$f(x) = x^2 + 2x + 1 \quad \text{is} \quad O(x^2)$$

proof: Find c and n_0 , then show inequality holds.

General strategy

- Start with n_0 : pick it so can estimate size of $f(n)$ for $n > n_0$.
 - ↳ often based on coefficients

- Then look for a constant c so inequality holds.

Example: Show $f(x) = 5x^2 + 25x + 3$ is $O(x^3)$.

We'll write $f(x) = O(g(x))$.

NOT AN EQUALITY!

• $x^2 + 2x + 1$ is $O(x^2)$

• $x^2 + 2x + 1$ is $O(x^3)$

Really, means $f(x) \in \{ \text{functions that are } O(g(x)) \}$

Polynomials are usually nice $\rightarrow$ but
Can do many functions!

Ex: $\sin x$ is $O(1)$.

Ex: n is $O(n \log_2 n)$.

Ex Show that n^2 is not $O(n)$

proof: harder! need to show

$$\neg (\exists c \exists n_0 \text{ s.t. } \forall n > n_0, n^2 \leq c \cdot n)$$



$\forall c$ and $\forall n_0$, $\exists$ some $n > n_0$ with

So:

Why do we care??

Back to pseudocode

We usually formalize the running time of an algorithm in terms of the number of atomic operations.

Examples:

Why count operations?

Generally, the other way to compare speed is benchmarking.

But this can vary even with identical algorithms:

Ideally: we'd like an abstract way to compare only the algorithms, regardless of context.

Why big-O?

The same "operation" in a language (ie a loop or function call) can use different low level operations, depending upon language, compiler, processor, etc.

Also, look at rates:

Table 7.3.1: Growth rates for different input sizes.

$f(n)$	$n = 10$	$n = 50$	$n = 100$	$n = 1000$	$n = 10000$	$n = 100000$
$\log_2 n$	$3.3 \mu s$	$5.6 \mu s$	$6.6 \mu s$	$10.0 \mu s$	$13.3 \mu s$	$16.6 \mu s$
n	$10 \mu s$	$50 \mu s$	$100 \mu s$	$1000 \mu s$	10 ms	0.1 s
$n \log_2 n$	0.03 ms	0.28 ms	0.66 ms	10.0 ms	0.133 s	1.67 s
n^2	0.1 ms	2.5 ms	10 ms	1 s	100 s	2.8 hours
n^3	1 ms	0.125 s	1 s	16.7 min	11.6 days	31.7 years
2^n	1.0 ms	35.7 years	$4.0 \times 10^{16} \text{ years}$	$3.4 \times 10^{287} \text{ years}$	$6.3 \times 10^{2996} \text{ years}$	*

This data assumes that a single instruction takes $1 \mu s$ to execute. A μs is a microsecond and is equal to 10^{-6} seconds. An ms is a millisecond and is equal to 10^{-3} seconds.

Consider:

$5n \log n$ versus n^2 , like when
sorting a list of 10,000 people

That limit definition

I did $f(n) = O(g(n))$ if $\exists n_0 \ \& \ c$ s.t.

$$\forall n > n_0, \quad f(n) \leq c \cdot g(n)$$

Take that inequality:

$$\text{So } f(n) = O(g(n)) \Leftrightarrow$$

Big Ω

Let f and g be functions from $\mathbb{Z}^+ \rightarrow \mathbb{R}^{\geq}$

We say f is $\Omega(g(n))$ if

$\exists$ positive constants c and n_0
such that $\forall n > n_0$,

$$f(n) \geq c \cdot g(n)$$

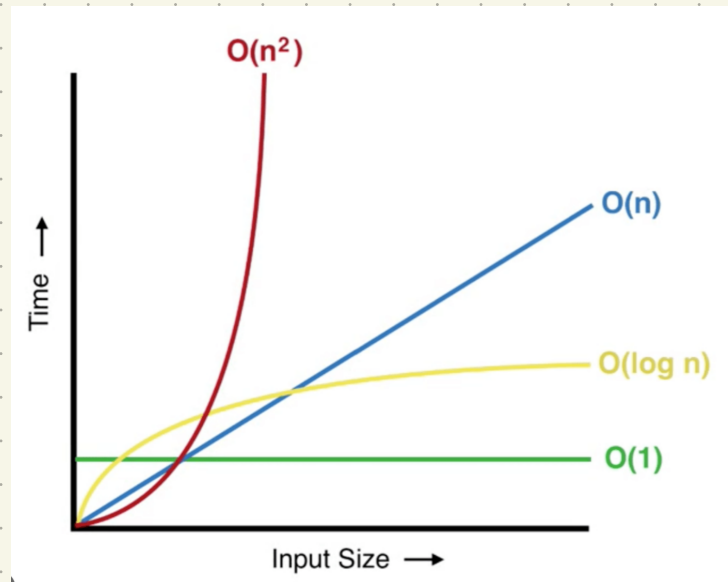
" f is big-omega of g "

Example: $5 \cdot n$ is $\Omega(n)$:

Big- Θ

If f is $O(g)$ and $\Omega(g)$ then f is $\Theta(g)$.

This gives a way to nicely "bucket"
function size:



Theorems

Let $p(n)$ be a degree k polynomial, so

$$p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

for constants a_i and $a_k \neq 0$.

Then $p(n) = \Theta(n^k)$.

Proof:

Big- Θ :

Big-O:

Using this theorem;

Give big- Θ estimates for:

- $f(x) = \frac{1}{8} x^5 + x^3 + 2$

- $f(x) = 20000 x^2 - 100000 x$

- $f(x) = \frac{2x^3}{3000} - x + 2$